

Benchmarking Code Correctness in Large Language Models:

A Failure-Mode Analysis Across Difficulty and Model Scale

CSCI-4364/6364 S26 — Machine Learning

Abenezer Golda(G30469085) | Abiy Mamo(G42938054) | Rufai Yakubu(G35886244)

Department of Computer Science, George Washington University

Abstract

Large language models are increasingly deployed as coding assistants, yet benchmarks like HumanEval report only aggregate pass rates that conceal where and how models fail. We constructed a 90-problem Python benchmark across three difficulty tiers (Easy, Medium, Hard), evaluated GPT-3.5-Turbo and CodeLlama-7B-Instruct using pass@1 and pass@3 with bootstrap confidence intervals, and produced a four-category failure taxonomy (Syntactic, Runtime, Algorithmic, Edge-case). Our results show that pass@1 does not decline monotonically with difficulty, that failure composition shifts across tiers with Algorithmic failures peaking at Medium, and that pass@3 preferentially recovers Syntactic and Runtime errors while Algorithmic failures persist. Cross-model Jaccard overlap is only 0.12–0.22, suggesting the models fail on largely distinct problems.

Github link:

Deliverables

- **Benchmark dataset:** 90 Python problems (30 per tier) with 630 validated test cases (3 standard, 2 boundary, 2 adversarial each), including 15 original contamination-free problems.
- **Evaluation harness:** Automated pipeline handling prompt formatting, API calls with retry/caching, sandboxed subprocess execution (5s timeout), hierarchical failure classification, and metric computation.
- **Analysis suite:** Six publication-quality charts (pass@k with CIs, taxonomy stacked bars, overlap heatmap, recovery rates, overall comparison, failure heatmap) and an auto-generated findings report.
- **Classifier validation tool:** Stratified 10% sample export with Cohen’s kappa scoring, per-category precision/recall/F1, and confusion matrix.
- **Reproducible repository:** One-command runner (run.sh), requirements.txt, and cached raw results for full reproduction.

Experiment Design

Research Questions. RQ1: Does pass@1 decline monotonically with difficulty? RQ2: Does failure composition shift across tiers? RQ3: Does pass@3 preferentially recover specific failure types? RQ4: Is there a model-scale effect on failure distribution?

Benchmark. 90 Python problems stratified into 30 Easy, 30 Medium, and 30 Hard using LeetCode’s community-verified taxonomy. Sources include HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021), and 15 original problems. Each problem has 7+ test cases: 3 standard inputs, 2 boundary conditions, and 2 adversarial edge cases designed to catch common implementation mistakes. The adversarial tests separate Algorithmic failures (wrong on standard inputs) from Edge-case failures (correct on standard but wrong on adversarial).

Design evolution. Two pivots occurred during the project. First, we shifted from HumanEval-only sourcing to mixed sourcing with 15 original problems after recognizing contamination risks highlighted by EvalPlus

(Liu et al., 2023). Second, we expanded the taxonomy from three categories to four, splitting “Wrong Output” into Algorithmic and Edge-case after pilot testing revealed qualitatively different failure modes.

Success criteria. Pass@1 and pass@3 via the unbiased estimator of Chen et al. (2021) with 95% bootstrap CIs (1,000 resamples). Failure proportions per (model, tier) cell. Cohen’s kappa ≥ 0.80 for classifier validation.

Methodology

Models Under Evaluation

We selected two models representing different points on the parameter-count spectrum to test whether model scale changes the composition of failures, not just their frequency:

Property	GPT-3.5-Turbo	CodeLlama-7B	Rationale
Parameters	~175B (estimated)	7B	25× scale gap
Access	OpenAI API	Together.ai API	Both OpenAI-compatible
Cost	~\$5 total	~\$0.50 total	Under \$7 combined
Training focus	General-purpose	Code-specialized	Tests specialization effect
Weights	Closed	Open (Roziere+, 2023)	Reproducibility anchor

Both models were queried at temperature 0.7 with max_tokens = 1024. Each model generated 3 independent solutions per problem, yielding 540 total API calls ($90 \times 2 \times 3$). Temperature 0.7 was chosen to balance diversity (needed for meaningful pass@3) against coherence.

Evaluation Pipeline

Stage 1 — Prompt Formatting. Each problem’s docstring specification is wrapped in model-specific prompt templates. For GPT-3.5-Turbo, we use the chat completion API with a system message instructing return of only the complete function definition. For CodeLlama, we use Together.ai’s OpenAI-compatible chat endpoint with an equivalent instruction. Prompt wording was calibrated on a 5-problem pilot to avoid common failure modes (CodeLlama omitting the def line; GPT-3.5 including explanatory text).

Stage 2 — Response Parsing. Raw model outputs are parsed into executable Python code via a multi-strategy extractor. The parser first attempts to extract code from markdown-fenced blocks, then searches for a def statement matching the entry point, and finally falls back to treating the entire response as code. Post-processing handles: missing def lines (reconstructed from the prompt), duplicate docstrings, indentation normalization, and trailing incomplete lines. This module was the most iteration-intensive: we went through four parser versions, as a parser that mishandles 5% of responses would silently inflate our Syntactic failure count.

Stage 3 — Sandboxed Execution. Each parsed solution is executed in an isolated Python subprocess against all 7+ tests. Safety measures include: a 5-second timeout for infinite loops; stdout/stderr capture for error classification; deep-copied inputs to prevent cross-test mutation; and JSON-formatted per-test results recording expected vs. actual output plus exception types.

Stage 4 — Failure Classification. The classifier applies a strict hierarchical decision tree ensuring mutual exclusivity. First, it checks whether the code compiles (Syntactic if SyntaxError, IndentationError, or NameError). Second, it checks for runtime exceptions during execution (Runtime if TypeError, IndexError, ValueError, ZeroDivisionError, Timeout, etc.). Third, it checks standard test results (Algorithmic if any standard test fails). Fourth, if all standard tests pass but boundary or adversarial tests fail, it assigns

Edge-case. This ordering guarantees every failure maps to exactly one category and that earlier, more severe failure modes take precedence.

Metrics Formulation

Our primary metric is $\text{pass}@k$, computed using the unbiased combinatorial estimator introduced by Chen et al. (2021): $\text{pass}@k = 1 - C(n-c, k) / C(n, k)$, where $n = 3$ (samples per problem), c = number of correct samples, and $C(a,b)$ is the binomial coefficient. This estimator avoids the upward bias that would result from simply checking whether any of k samples passes.

For confidence intervals, we use non-parametric bootstrap resampling: for each (model, tier) combination, we resample the 30 per-problem $\text{pass}@k$ values with replacement 1,000 times and report the 2.5th and 97.5th percentiles of the resampled means. Secondary metrics include failure rate per category per (model, tier) cell, cross-model Jaccard similarity, and $\text{pass}@3$ recovery rate per failure category.

Results

RQ1: Pass@1 Across Difficulty Tiers

Table 1 presents $\text{pass}@1$ for each (model, tier) combination with 95% bootstrap confidence intervals. Contrary to our initial hypothesis, $\text{pass}@1$ does not decline monotonically with difficulty for either model. Both models show the lowest $\text{pass}@1$ at Medium difficulty rather than Hard.

Tier	GPT-3.5 $\text{pass}@1$	CodeLlama $\text{pass}@1$	Difference
Easy	0.878 [0.77, 0.96]	0.800 [0.68, 0.91]	+0.078
Medium	0.767 [0.64, 0.88]	0.744 [0.62, 0.86]	+0.023
Hard	0.844 [0.73, 0.94]	0.822 [0.70, 0.93]	+0.022

Table 1: $\text{Pass}@1$ with 95% bootstrap CIs. GPT-3.5 outperforms CodeLlama by 2–8 pp but confidence intervals overlap at all tiers.

This non-monotonic pattern likely reflects tier composition: Medium problems require novel algorithmic reasoning (backtracking, DP, sliding windows) while some Hard problems test well-known patterns (edit distance, topological sort) that appear extensively in training corpora. GPT-3.5 outperforms CodeLlama across all tiers but the gap is modest and overlapping CIs preclude statistical significance claims. $\text{Pass}@3$ reaches 0.97–1.00 for both models, indicating most failures are recoverable with multiple samples.

RQ2: Failure Composition Across Tiers

Table 2 breaks down the failure taxonomy by model and tier. The compositional shifts are the most actionable finding of this study.

Model / Tier	Syntactic	Runtime	Algorithmic	Edge-case
GPT-3.5 Easy	9%	82%	9%	0%
GPT-3.5 Med	33%	43%	24%	0%
GPT-3.5 Hard	29%	64%	7%	0%
CLlama Easy	17%	72%	11%	0%
CLlama Med	17%	48%	35%	0%
CLlama Hard	25%	69%	6%	0%

Table 2: Failure type proportions by (model, tier). Algorithmic failures peak at Medium for both models.

Runtime errors dominate at Easy and Hard tiers (60–82% of failures), while Medium shows a more balanced distribution with the highest proportion of Algorithmic failures (24–35%). Edge-case failures — solutions that pass all standard tests but fail on adversarial inputs — are rare across all conditions. This is an encouraging finding for practitioners: when models get the core algorithm wrong, they tend to fail obviously on standard tests rather than producing subtly incorrect solutions that would pass casual review. Even modest test coverage would catch the vast majority of model errors.

RQ3: Pass@3 Recovery Analysis

We analyzed which failure types are recovered when sampling three times instead of once. Syntactic and Runtime errors are recovered at 92–100% rates, while Algorithmic failures recover at only 64–83%. This makes intuitive sense: syntax errors and runtime crashes are high-variance stochastic flukes (the model sometimes generates a missing colon or an off-by-one index, sometimes not), while Algorithmic failures indicate consistent misunderstanding of the problem logic. When a model does not know how to solve a problem, sampling more times is unlikely to help.

Practical implication: deployment systems using multi-sample aggregation (e.g., generating 3 candidates and running against a test suite) should expect significant gains on surface-level errors but limited improvement on core reasoning failures. The marginal value of additional samples is concentrated in the easiest-to-fix error categories.

RQ4: Model-Scale Effect on Failure Distribution

Both models show similar aggregate failure type distributions (Runtime ~60%, Syntactic ~20%, Algorithmic ~17%). However, the cross-model Jaccard similarity for failure sets is only 0.12–0.22, meaning they fail on largely distinct problems despite producing similar type-level patterns. This low overlap suggests that model scale affects which specific problems a model struggles with, even if it does not dramatically change the kinds of errors it makes.

Ensemble implication: an ensemble of GPT-3.5 and CodeLlama would likely outperform either model alone, because their failure sets are nearly disjoint. A simple “run both, pick the one that passes tests” strategy would recover most of the failures unique to each model.

Classifier Validation

Manual review of a stratified 10% sample showed strong agreement with the automated classifier. The main disagreement was at the Algorithmic vs. Edge-case boundary: in a few cases, a solution produced the correct output on standard tests by coincidence (e.g., returning a hardcoded value that happened to match) and was classified as Edge-case when the reviewer judged it should be Algorithmic. These borderline cases do not affect aggregate findings.

Lessons Learned

Benchmark construction is the hardest part. Writing adversarial test cases requires anticipating how models fail. Our initial suites contained bugs where the “expected” output was wrong — caught only by validating all 630 canonical solutions before model evaluation.

The response parser matters more than you think. Models return code in wildly inconsistent formats. A parser working on 95% of responses silently corrupts the rest into Syntactic misclassifications. We iterated through four versions before settling on the multi-strategy approach.

Mock mode and aggressive caching save time and money. A MockAPICaller let us debug six pipeline bugs without API costs. Caching every raw response to disk meant we never re-ran calls when we found a parser bug in Week 3. Total cost: under \$7.

Attribution

- **Abenezer Golda:** Project lead. Designed the benchmark, wrote the 15 original problems, implemented the failure classifier, ran the final evaluation, and wrote the report.
- **Abiy Mamo:** Built the evaluation harness (API caller, response parser, sandboxed executor), implemented mock mode, and handled all API integration and caching.
- **Rufai Yakubu:** Implemented the metrics module (pass@k, bootstrap CIs, cross-model overlap), built the visualization suite, performed classifier manual validation, and created the GitHub repository.

References

- Chen, M., et al. (2021). Evaluating Large Language Models Trained on Code.
- Austin, J., et al. (2021). Program Synthesis with Large Language Models.
- Liu, J., Xia, C.S., Wang, Y. & Zhang, L. (2023). Is Your Code Generated by ChatGPT Really Correct?
- Jain, N., et al. (2024). LiveCodeBench: Holistic and Contamination Free Evaluation of LLMs for Code.
- Nguyen, N. & Nadi, S. (2022). An Empirical Evaluation of GitHub Copilot’s Code Suggestions. , pp. 1–11.
- Pearce, H., et al. (2022). Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions. , pp. 754–768.
- Roziere, B., et al. (2023). Code Llama: Open Foundation Models for Code.
- Efron, B. & Tibshirani, R.J. (1993). Chapman and Hall/CRC.
- Landis, J.R. & Koch, G.G. (1977). The Measurement of Observer Agreement for Categorical Data. , 33(1), pp. 159–174.
- GitHub. (2023). The State of the Octoverse 2023. <https://octoverse.github.com/>.